Jenkins Day 1 — What is Jenkins?

1. What is Jenkins?

Jenkins is an open-source automation server used to automate software development processes such as:

- Building applications
- Running unit/integration tests
- Code quality checks
- Security scanning
- Creating Docker images
- Deploying applications
- Infrastructure automation
- Kubernetes deployments

Jenkins is primarily used to implement **CI/CD pipelines**.

Simple example

Without Jenkins:

```
Developer
   ↓
Git Push
   ↓
Developer manually builds application
   ↓
Developer manually runs tests
   ↓
Developer creates Docker image
   ↓
Developer manually deploys
```

With Jenkins:

```
Developer
   ↓
Git Push
   ↓
Jenkins
   ↓
Build
   ↓
```

```
Test
   ↓
Code Quality
   ↓
Security Scan
   ↓
Docker Build
   ↓
Push Image
   ↓
Deploy
```

The goal is to **automate repetitive tasks and provide fast feedback**.

---

2. Why Jenkins?

Organizations use Jenkins because it can automate the complete software delivery lifecycle.

Major benefits

1. Automation
2. Faster releases
3. Consistent deployments
4. Early bug detection
5. Integration with many DevOps tools
6. Pipeline as Code
7. Distributed builds
8. Plugin ecosystem
9. Support for Docker and Kubernetes
10. Easy integration with Git-based workflows

Common Jenkins integrations

```
Jenkins
 ├── GitHub / GitLab
 ├── Maven
 ├── Gradle
 ├── Python
 ├── Docker
 ├── Kubernetes
 ├── AWS
 ├── Terraform
 ├── Ansible
 ├── SonarQube
```

```
├── Trivy
├── Nexus
└── JFrog Artifactory
```

---

3. What is CI?

CI = Continuous Integration

Continuous Integration means developers frequently integrate their code into a shared repository and automatically validate the changes.

CI flow

```
Developer
    ↓
git push
    ↓
Git Repository
    ↓
Jenkins Trigger
    ↓
Checkout Code
    ↓
Build
    ↓
Unit Tests
    ↓
Code Quality
    ↓
Security Scan
    ↓
Build Result
```

Example

A Python developer pushes:

```
git push origin feature/login
```

Jenkins detects the change and executes:

```
Checkout
   ↓
Create virtual environment
   ↓
Install dependencies
   ↓
Run pytest
   ↓
Run linting
   ↓
Run security scan
```

If everything passes:

```
BUILD SUCCESS
```

If tests fail:

```
BUILD FAILURE
```

The developer gets immediate feedback.

---

4. Continuous Delivery vs Continuous Deployment

These three concepts are commonly confused.

Continuous Integration

Automatically:

```
Code → Build → Test → Validate
```

The application is validated continuously.

Continuous Delivery

The application is automatically built, tested, packaged, and made **ready for production deployment**.

```
Code
 ↓
Build
 ↓
Test
 ↓
Security Scan
 ↓
Package
 ↓
Artifact Repository
 ↓
READY FOR PRODUCTION
```

Production deployment may still require manual approval.

Example:

```
Jenkins
   ↓
Build
   ↓
Test
   ↓
Docker Image
   ↓
ECR
   ↓
Manual Approval
   ↓
Production
```

Continuous Deployment

The deployment to production is completely automated.

```
Code
 ↓
Build
 ↓
Test
 ↓
```

```
Security Scan
 ↓
Docker Image
 ↓
Registry
 ↓
Deploy
 ↓
Production
```

No manual approval is required.

---

5. CI vs Continuous Delivery vs Continuous Deployment

Feature	CI	Continuous Delivery	Continuous Deployment
Build	Yes	Yes	Yes
Testing	Yes	Yes	Yes
Security scanning	Usually	Usually	Usually
Artifact creation	Usually	Yes	Yes
Production ready	No	Yes	Yes
Automatic production deployment	No	Usually no	Yes
Manual approval	Not applicable	Possible	Usually no

Easy way to remember

```
CI:
"Is my code working?"

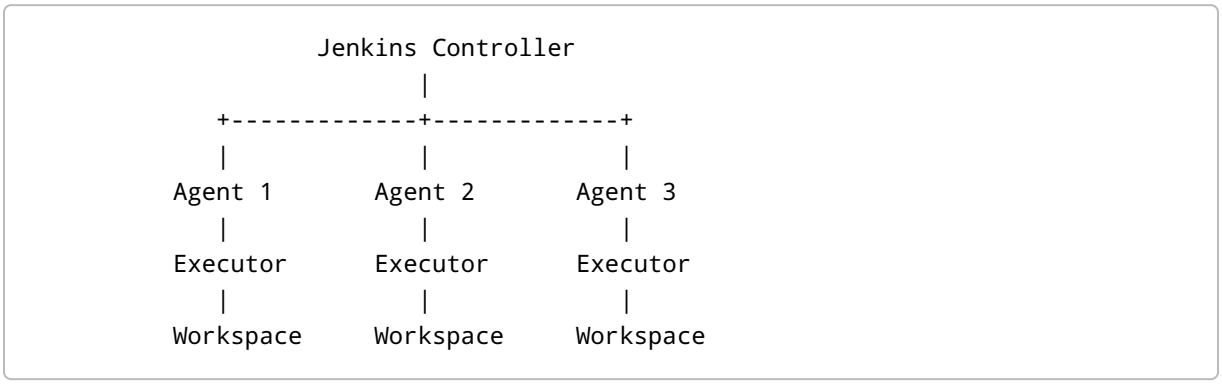
Continuous Delivery:
"Is my application ready to deploy?"

Continuous Deployment:
"Deploy it automatically."
```

---

6. Jenkins Architecture

Jenkins commonly follows a controller-agent architecture.



The controller manages Jenkins.

Agents perform the actual workload.

---

7. Jenkins Controller

The **Jenkins Controller** is the central Jenkins component.

It is responsible for:

- Managing Jenkins configuration
- Managing jobs
- Scheduling builds
- Managing credentials
- Managing plugins
- Managing agents
- Storing Jenkins metadata
- Managing build information
- Providing the Jenkins UI
- Coordinating pipeline execution

Important point

The controller should preferably **not perform heavy build workloads** in a production environment.

Instead:

```
Controller
    ↓
Schedules job
    ↓
Agent
    ↓
Performs build
```

---

8. Jenkins Agent

A **Jenkins Agent** is a machine/container that executes Jenkins tasks.

Agents can be:

- Linux servers
- Windows servers
- Docker containers
- Kubernetes pods
- Cloud instances
- Virtual machines

Example:

```
Jenkins Controller
        |
        +---- Python Agent
        |
        +---- Docker Agent
        |
        +---- Kubernetes Agent
        |
        +---- Windows Agent
```

Why use agents?

Suppose you have:

```
100 builds/day
```

One machine may not be sufficient.

You can create multiple agents:

```
Controller
   |
   +---- Agent 1 → Python builds
   |
   +---- Agent 2 → Java builds
   |
   +---- Agent 3 → Docker builds
   |
   +---- Agent 4 → Windows builds
```

This allows distributed execution.

---

9. Executor

An **Executor** is a slot that allows a Jenkins agent to execute a task.

Example:

```
Agent
 ├── Executor 1 → Build A
 ├── Executor 2 → Build B
 └── Executor 3 → Build C
```

If an agent has:

```
4 executors
```

it can potentially execute four tasks concurrently, assuming resources and scheduling allow it.

Important interview point

Agent = machine/environment

Executor = execution slot on that agent

---

10. Workspace

A **Workspace** is the directory where Jenkins checks out and works with application source code during a build.

Example:

```
/var/lib/jenkins/workspace/my-python-app/
```

Inside:

```
my-python-app/
├── app/
├── tests/
├── requirements.txt
├── Dockerfile
└── Jenkinsfile
```

Jenkins may execute:

```
git clone ...
pip install -r requirements.txt
pytest
docker build .
```

inside the workspace.

Important

Workspace is generally **not your artifact repository**.

For example:

```
Workspace
   ↓
Temporary build working area

Artifact Repository
   ↓
Long-term storage of build artifacts
```

---

11. Jenkins Job

A **Job** is a Jenkins task/configuration that defines what Jenkins should execute.

Examples:

```
Python-Build
Docker-Build
Production-Deploy
Database-Migration
```

A job may contain:

```
Source Code
Build Steps
Test Steps
Environment Variables
Credentials
Triggers
Post-build Actions
```

Modern Jenkins commonly uses **Pipeline jobs**.

---

12. Jenkins Build

A **Build** is one execution/run of a Jenkins job.

Suppose you have:

```
Job: Python-Application
```

Executions may be:

```
Build #1
Build #2
Build #3
Build #4
```

Each build represents a specific execution.

Example:

```
Build #25
Commit: abc123
Status: SUCCESS
Duration: 4 minutes
```

---

13. Jenkins Plugins

Plugins extend Jenkins functionality.

Jenkins itself provides the core automation engine, while plugins integrate it with external tools.

Examples:

```
Git Plugin
Docker Plugin
Kubernetes Plugin
Pipeline Plugin
Credentials Plugin
Slack Plugin
SonarQube Plugin
AWS Plugins
```

Example

Without integration:

```
Jenkins → Git
```

Git plugin enables Jenkins to interact with Git repositories.

Similarly:

```
Jenkins → Kubernetes
Jenkins → SonarQube
Jenkins → Docker
```

through appropriate integrations/plugins.

Important interview question

Why does Jenkins have so many plugins?

Because Jenkins is designed to be extensible and integrate with many development, testing, infrastructure, cloud, and deployment tools.

---

14. Jenkins Credentials

Jenkins Credentials securely stores authentication information used by pipelines/jobs.

Examples:

- Username/password
- SSH private keys
- API tokens
- Secret text
- Certificates
- Cloud credentials

Example:

```
Jenkins
   |
   +---- Git Credentials
   |
   +---- Docker Registry Credentials
   |
   +---- AWS Credentials
   |
   +---- Kubernetes Credentials
```

Example

Instead of writing:

```
git password = "mypassword"
```

you store the secret in Jenkins Credentials.

Then reference it securely.

Example:

```
withCredentials([
    usernamePassword(
        credentialsId: 'docker-creds',
        usernameVariable: 'USERNAME',
        passwordVariable: 'PASSWORD'
    )
]) {
    sh 'docker login -u $USERNAME -p $PASSWORD'
}
```

Never hard-code secrets in a Jenkinsfile.

---

15. Jenkins Pipeline

A **Pipeline** defines the complete CI/CD workflow as code.

Example:

```
Checkout
   ↓
Build
   ↓
Test
   ↓
SonarQube
   ↓
Security Scan
   ↓
Docker Build
   ↓
Push Image
   ↓
Deploy
```

A Jenkins pipeline is commonly stored in a file called:

```
Jenkinsfile
```

Example:

```
pipeline {

    agent any

    stages {

        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Install') {
            steps {
                sh 'pip install -r requirements.txt'
            }
        }

        stage('Test') {
            steps {
                sh 'pytest'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t myapp:latest .'
            }
        }
    }
}
```

---

16. Declarative vs Scripted Pipeline

Jenkins supports two major Pipeline syntaxes.

Declarative Pipeline

Example:

```
pipeline {
    agent any
```

```
    stages {
        stage('Build') {
            steps {
                sh 'python app.py'
            }
        }
    }
}
```

Declarative Pipeline is generally easier to read and maintain.

Scripted Pipeline

Example:

```
node {
    stage('Build') {
        sh 'python app.py'
    }
}
```

Scripted Pipeline provides more programming flexibility.

Interview answer

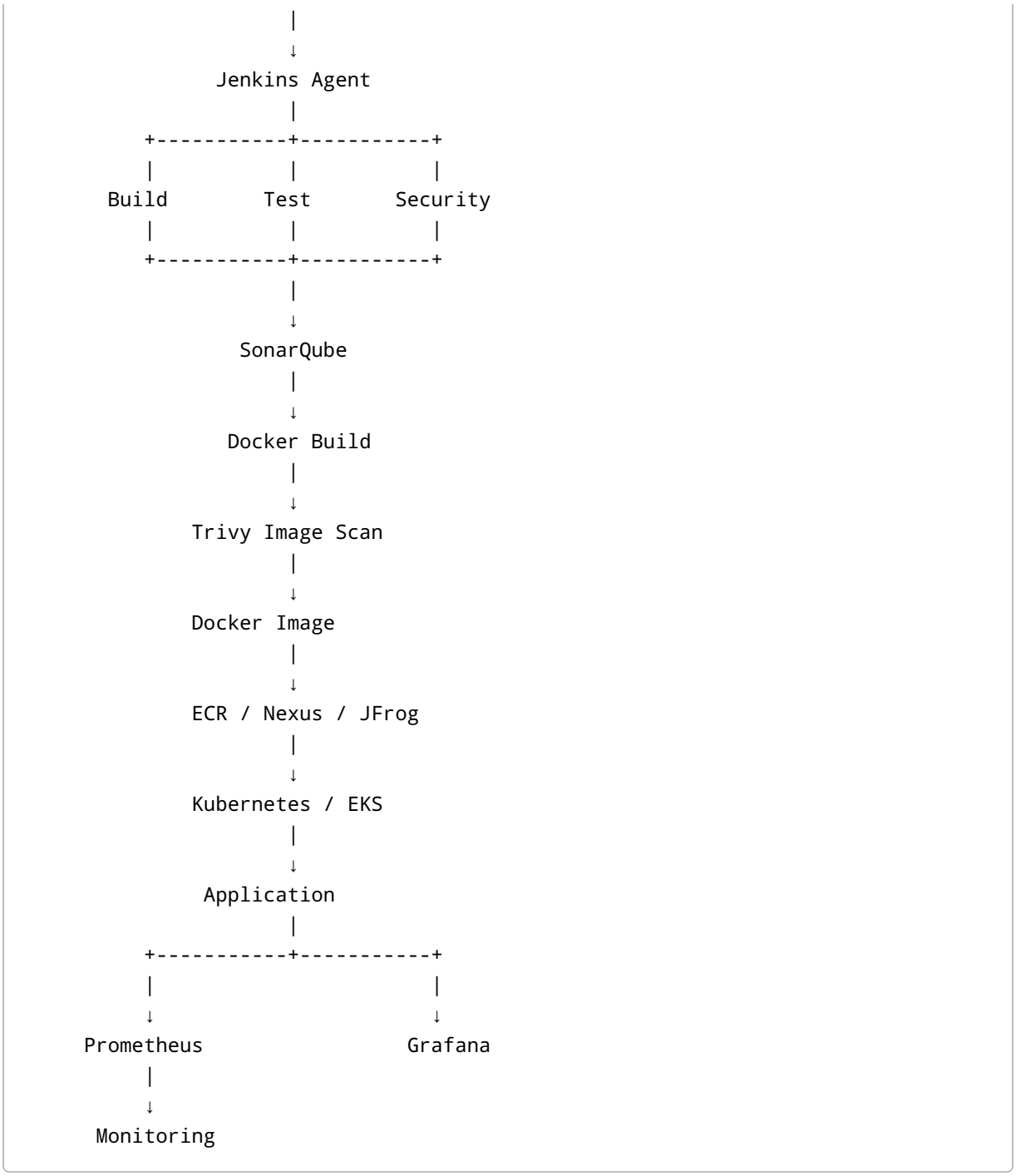
For most enterprise CI/CD pipelines, **Declarative Pipeline** is commonly preferred because it provides a structured syntax and makes pipelines easier to maintain.

---

17. Real-World CI/CD Architecture for Python

A typical production architecture could look like:

```
                Developer
                    |
                    | git push
                    ↓
              GitHub / GitLab
                    |
                    | Webhook
                    ↓
            Jenkins Controller
```



---

18. Real-World Python CI/CD Pipeline

Suppose we have:

```
Python FastAPI Application
```

Repository:

```
my-fastapi-app/
├── app/
├── tests/
├── requirements.txt
├── Dockerfile
├── Jenkinsfile
└── README.md
```

Pipeline:

```
Developer Push
      ↓
GitHub
      ↓
Jenkins Webhook
      ↓
Checkout
      ↓
Install Dependencies
      ↓
Unit Tests
      ↓
Lint
      ↓
SonarQube
      ↓
Dependency Scan
      ↓
Docker Build
      ↓
Trivy Scan
      ↓
Push Docker Image
      ↓
Deploy to Kubernetes
      ↓
Smoke Test
      ↓
Production
```

19. Example Jenkins Pipeline

```
pipeline {

    agent any

    environment {
        IMAGE_NAME = 'my-python-app'
        IMAGE_TAG  = "${BUILD_NUMBER}"
    }

    stages {

        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Install Dependencies') {
            steps {
                sh '''
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install -r requirements.txt
                '''
            }
        }

        stage('Unit Tests') {
            steps {
                sh '''
                    . venv/bin/activate
                    pytest
                '''
            }
        }

        stage('Lint') {
            steps {
                sh '''
                    . venv/bin/activate
                    flake8 .
                '''
            }
```

```
        }

        stage('Docker Build') {
            steps {
                sh '''
                    docker build
                    -t ${IMAGE_NAME}:${IMAGE_TAG} .
                '''
            }
        }

        stage('Security Scan') {
            steps {
                sh '''
                    trivy image ${IMAGE_NAME}:${IMAGE_TAG}
                '''
            }
        }

        stage('Deploy') {
            steps {
                sh '''
                    kubectl apply -f k8s/
                '''
            }
        }
    }
}
```

---

20. Webhook-Based Jenkins Trigger

A common production flow is:

```
Developer
   |
   | git push
   ↓
GitHub
   |
   | Webhook
   ↓
Jenkins
   |
```

```
    ↓
Pipeline
```

The webhook prevents Jenkins from continuously polling Git.

Example

Developer pushes:

```
git push origin main
```

GitHub sends an HTTP request to Jenkins.

Jenkins receives the event and starts the pipeline.

---

21. Jenkins Controller vs Agent

Component	Responsibility
Controller	Manages Jenkins
Agent	Executes workloads
Executor	Execution slot
Workspace	Build working directory
Job	Jenkins task/configuration
Build	Execution of a job
Plugin	Extends Jenkins functionality
Credential	Secure authentication information
Pipeline	CI/CD workflow

Easy interview explanation

> Jenkins Controller manages and schedules Jenkins workloads. Agents execute those workloads. Executors are the available execution slots on agents, and the workspace is the directory where the build operates.

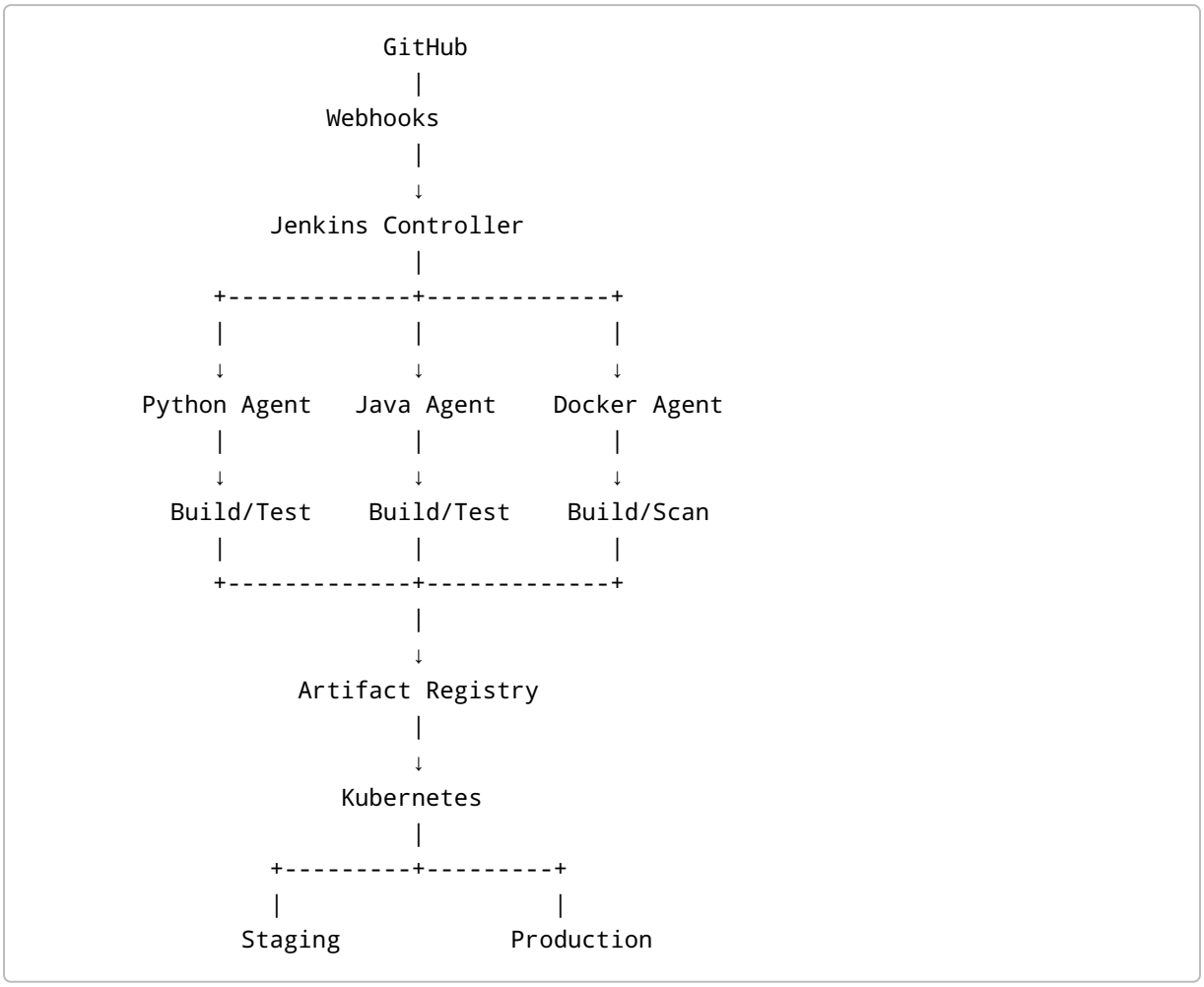
---

22. Example Enterprise Architecture

Imagine a company has:

```
100 developers
10 Python applications
20 Java applications
5 frontend applications
```

A scalable Jenkins architecture might be:



---

23. Assignment — Design CI/CD Architecture for a Python Application

Scenario

You have a Python FastAPI application.

Requirements:

- GitHub repository
- Jenkins CI/CD
- Unit testing
- Code quality analysis
- Security scanning
- Docker image
- Container registry
- Kubernetes deployment
- Monitoring

Your architecture

Design:

```
Developer
    |
    ↓
GitHub
    |
    ↓
Webhook
    |
    ↓
Jenkins Controller
    |
    ↓
Jenkins Agent
    |
    +---- Checkout
    |
    +---- Install Dependencies
    |
    +---- Unit Tests
    |
    +---- Lint
```

```
    |
    +---- SonarQube
    |
    +---- Security Scan
    |
    +---- Docker Build
    |
    +---- Trivy Scan
    |
    ↓
Container Registry
    |
    ↓
Kubernetes / EKS
    |
    +---- Deployment
    |
    +---- Service
    |
    +---- Ingress
    |
    ↓
Application
    |
    +---- Prometheus
    |
    +---- Grafana
```

Questions you should answer

1. Where does the source code live?
2. How does Jenkins know that code has changed?
3. Where does the pipeline execute?
4. Which Jenkins agent should execute the build?
5. Where are credentials stored?
6. How do you run Python unit tests?
7. How do you perform code quality analysis?
8. How do you scan the Docker image?
9. Where do you store the Docker image?
10. How does Jenkins deploy to Kubernetes?
11. How do you monitor the deployed application?
12. What happens if the tests fail?
13. What happens if the security scan fails?
14. How would you implement production approval?
15. How would you roll back a failed deployment?

---

24. Day 1 Interview Questions

Q1. What is Jenkins?

Answer:

Jenkins is an open-source automation server used to automate CI/CD workflows such as building, testing, scanning, packaging, and deploying applications.

Q2. What is CI?

Answer:

Continuous Integration is the practice of frequently integrating code changes into a shared repository and automatically building and testing those changes to detect problems early.

Q3. Difference between Continuous Delivery and Continuous Deployment?

Answer:

Continuous Delivery automatically prepares software for production and may require manual approval before deployment. Continuous Deployment automatically deploys validated changes to production without manual approval.

Q4. What is a Jenkins Controller?

Answer:

The Jenkins Controller is the central Jenkins component responsible for configuration, scheduling, job management, plugin management, credentials, and coordinating agents.

Q5. What is a Jenkins Agent?

Answer:

A Jenkins Agent is a machine, VM, container, or Kubernetes pod that executes Jenkins workloads assigned by the controller.

Q6. What is an Executor?

Answer:

An executor is an execution slot on a Jenkins node/agent that allows Jenkins to run a build or pipeline task.

Q7. What is a Jenkins Workspace?

Answer:

A workspace is the directory allocated to a job where Jenkins checks out source code and performs build, test, and other pipeline operations.

Q8. What is a Jenkins Job?

Answer:

A Jenkins Job defines a task or automation process that Jenkins can execute.

Q9. What is a Jenkins Build?

Answer:

A build is a specific execution of a Jenkins job.

Q10. What are Jenkins plugins?

Answer:

Plugins extend Jenkins functionality and allow Jenkins to integrate with tools such as Git, Docker, Kubernetes, SonarQube, AWS, and notification systems.

Q11. How do you securely store passwords in Jenkins?

Answer:

Use the Jenkins Credentials system and reference credentials through credential IDs in pipelines instead of hard-coding secrets.

Q12. What is a Jenkinsfile?

Answer:

A Jenkinsfile is a Pipeline-as-Code file that defines the CI/CD workflow and is normally stored alongside the application source code.

---

25. Important Day 1 Terminology

Remember these terms:

```
Jenkins
   ↓
Controller
   ↓
Agent
   ↓
Executor
   ↓
Workspace
   ↓
Job
   ↓
Build
   ↓
Pipeline
   ↓
Jenkinsfile
   ↓
Plugin
   ↓
Credential
```

And remember the CI/CD flow:

```
Code
 ↓
Build
 ↓
Test
 ↓
Quality
 ↓
```

```
Security
 ↓
Package
 ↓
Artifact
 ↓
Deploy
 ↓
Monitor
```

---

26. Quick Revision

Jenkins

Automation server for CI/CD.

CI

Automatically build and test code changes.

Continuous Delivery

Keep software production-ready, usually with an approval step before production.

Continuous Deployment

Automatically deploy validated changes to production.

Controller

Manages Jenkins.

Agent

Executes workloads.

Executor

Execution slot.

Workspace

Build working directory.

Job

Jenkins task.

Build

Execution of a job.

Plugin

Extends Jenkins functionality.

Credentials

Securely stores authentication information.

Jenkinsfile

Pipeline-as-Code definition.

Day 1 Practical Tasks

- [ ] Install Jenkins locally or on an Ubuntu VM
- [ ] Open Jenkins UI
- [ ] Create your first Jenkins job
- [ ] Create a Pipeline job
- [ ] Connect Jenkins to GitHub
- [ ] Configure a webhook
- [ ] Create a Jenkinsfile
- [ ] Run a Python application build
- [ ] Run `pytest`
- [ ] View Jenkins console logs
- [ ] Configure Jenkins credentials
- [ ] Create a Jenkins agent
- [ ] Understand executor configuration
- [ ] Inspect the Jenkins workspace
- [ ] Design the Python CI/CD architecture from the assignment

Day 1 Goal

By the end of Day 1, you should be able to explain:

> **"What is Jenkins, why organizations use it, how the Jenkins Controller-Agent architecture works, what Jobs/Builds/Executors/Workspaces are, how Jenkins handles credentials and plugins, and how Jenkins fits into a real-world CI/CD pipeline."**